Object Oriented Programming(C++)
(CSE 1211)

Object & Class

Md. Mahfuzur Rahman
Lecturer,
Prime University

Object

An Object is an identifiable entity with some characteristics and behavior. An Object is an instance of a Class or Class type variable.

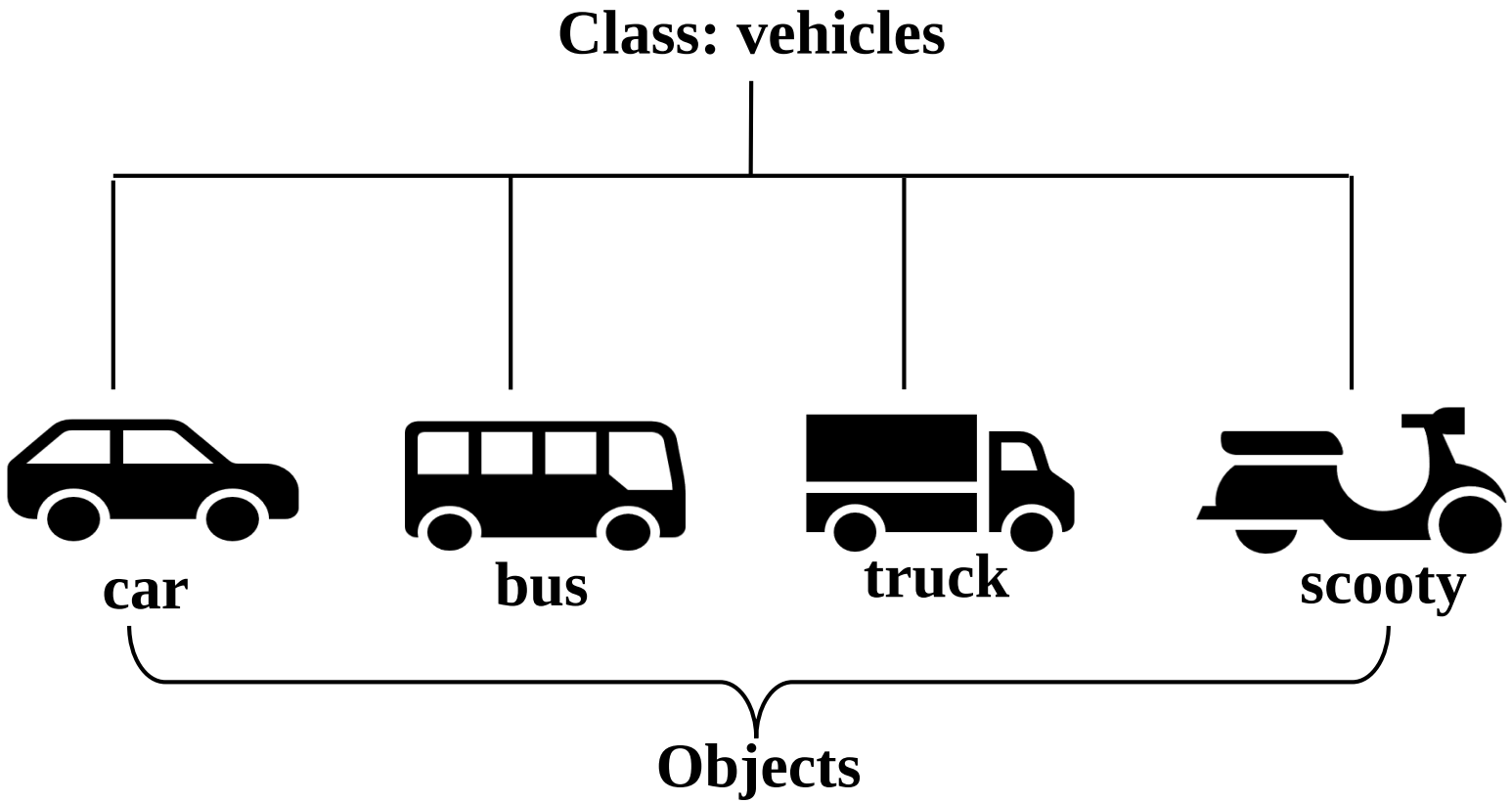
attributes - name, model, color, wheel, price, etc.

behavior - speed, engine, mileage etc.

Class

A Class is a user-defined data type that has data members and member functions. It is a collection of objects of similar type. It is a collection of data and member functions that manipulate data. The data components of class are called data members and functions that manipulate the data are called member functions. It can also called as blue print or prototype that defines the variables and functions common to all objects of certain kind. It is also known as user defined data type or ADT(abstract data type) A class is declared by the keyword **class**.

Object and Class



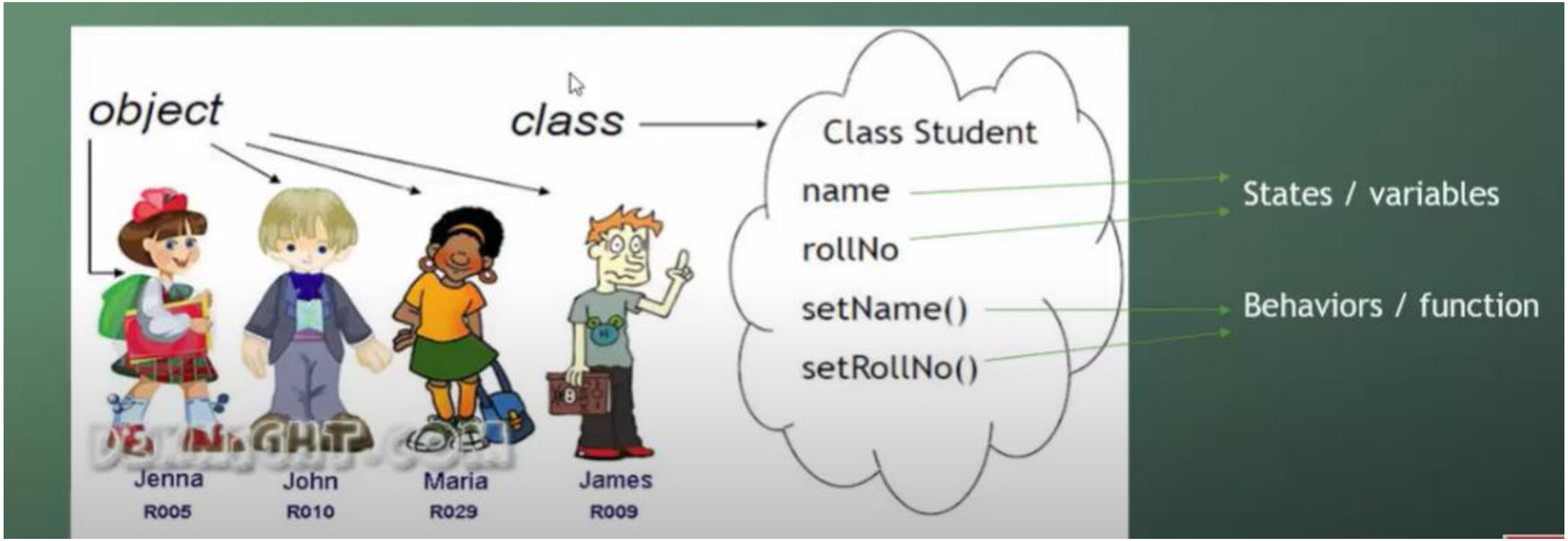

```
class vehicles
{
char name[10];
char model[10];
char color[10];
int wheel;
double price;

speed();
engine();
mileage();
};
```

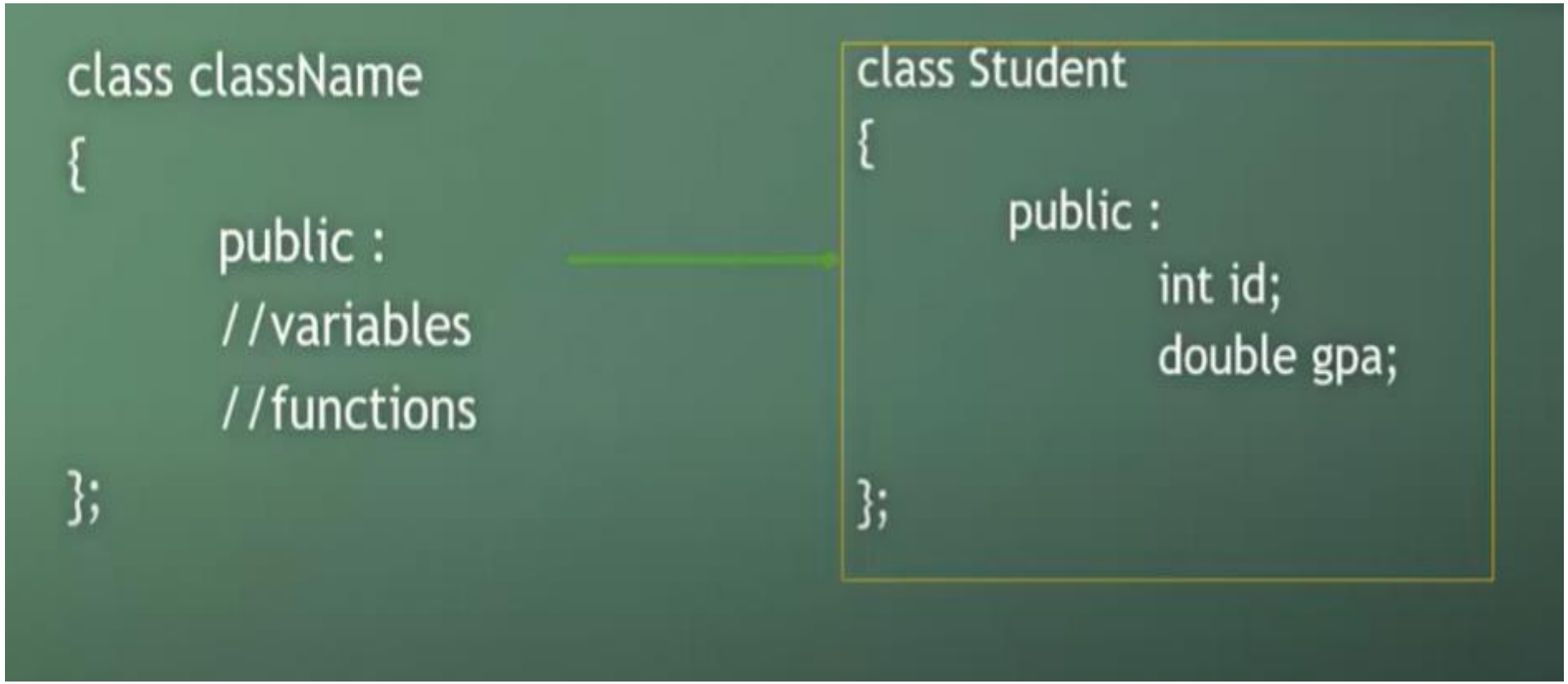
data members

member function

Class and Object



Class Declaration



Specifying a class

A class is a way to bind the data and its associated functions together. It allows the data (or, functions) to be hidden, if necessary, from external use. When defining a class, we are creating a new abstract data type that can be treated like any other built in data type. Generally, a class specification has two parts:

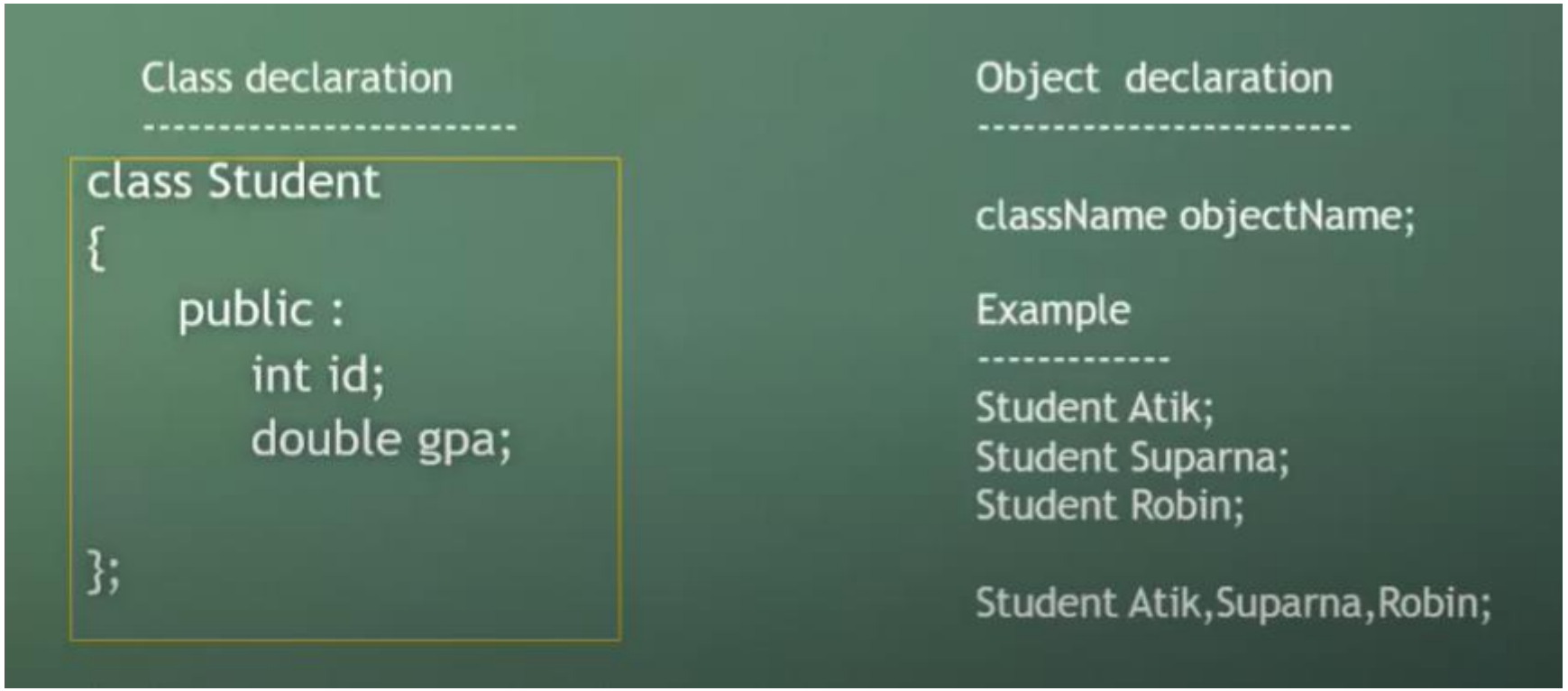
i) Class declaration
ii) Class function definitions

i) Class declaration

The class declaration describes the type scope of its members. The general form of a class declaration is:

class class_name
{
Access Specifier:
variable declarations
Access Specifier:
function declarations
};

Object Declaration



Access Specifier

Access specifier or access modifiers are the labels that specify type of access given to members of a class. These are used for data hiding. These are also called as visibility modes. There are three types of access specifiers

1.private
2.public
3.protected

1 Private:

- If the data members are declared as private access then they cannot be accessed from other functions outside the class.
- It can only be accessed by the functions declared within the class.
- It is declared by the key word **private**.

2 public:

- If the data members are declared public access then they can be accessed from other functions out side the class.
- It is declared by the key word **public**.

3. protected

- The protected keyword is used to create protected members (data and function).
- The protected members can be accessed within the class and from the derived class.
- The access level of protected declaration lies between public and private. This access specifier is used at the time of inheritance

Specifiers	Same Class	Derived Class	Outside Class
public	Yes	Yes	Yes
private	Yes	No	No
protected	Yes	Yes	No

Notes:

- If no access specifier is specified then it is treated by default as private

Access Modifiers in C++

The main advantage of data hiding is, you can change logic and representation of class members without modifying the codes of its users as it was not directly accessible.

1. Public Specifier: Public members of a class in C++ can be accessed from anywhere in the program. This means that both other classes and functions can directly use the data members and member functions marked as public. To access them, the dot (.) operator is used along with the object of the class.

2. Private Specifier: The class members declared as ***private*** can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class. Only the member functions or the friend function/friend class are allowed to access the private data members of the class.

3. Protected Specifier: The protected access modifier is similar to the private access modifier in the sense that it can't be accessed outside of its class unless with the help of a friend class. The difference is that the class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

Class and Object

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;
    };

int main()
{
    Student alif;

    alif.id = 101;
    alif.cgpa = 3.80;

    cout << alif.id << " " << alif.cgpa << endl;

    return 0;
}
```

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;
    };

int main()
{
    Student alif;

    alif.id = 101;
    alif.cgpa = 3.80;
    cout << alif.id << " " << alif.cgpa << endl;

    Student esha;

    esha.id = 102;
    esha.cgpa = 3.90;
    cout << esha.id << " " << esha.cgpa << endl;

    return 0;
}
```

Class and Object

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;

            void display()
            {
                cout << id << " " << cgpa << endl;
            }
    };

int main()
{
    Student alif;

    alif.id = 101;
    alif.cgpa = 3.80;
    alif.display();

    Student esha;

    esha.id = 102;
    esha.cgpa = 3.90;
    esha.display();

    return 0;
}
```

```
101 3.8
102 3.9

Process returned 0 (0x0)   execution time : 0.030 s
Press any key to continue.
```

Class and Object

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;

            void input(int x, double y)
            {
                id = x;
                cgpa = y;
            }

            void display()
            {
                cout << id << " " << cgpa << endl;
            }
    };
```

```cpp
int main()
{
    Student alif,esha;

    alif.input(101,3.80);
    alif.display();

    esha.input(102,3.90);
    esha.display();

    return 0;
}
```

ii) Class function definitions

Member functions can be defined in two places:

- Inside the class definition
- Outside the class definition

Inside the class definition:

```
class item
{
int number;
float cost;
public:
void getdata(int a, float b)
void putdata(void)
{
cout<<number<<endl;
cout<<cost<<endl;
}
};
```

- **Outside the class definition**

Syntax:

```
return_type class_name :: function_name(argument declaration)
{
  //function body
}
```

Example,

```
void item :: getdata(int a, float b)
{
number = a;
cost = b;
}

void item :: putdata(void)
{
cout<< “Number: “<<number<<endl;
cout<<“Cost: “<<cost<<endl;
}
```

Write a program to find area of rectangle

```
#include<iostream>
using namespace std;
class rectangle
{
int L,B;
public:
void get_data();
int area();
};
void rectangle::get_data()
{
cout<<”Enter Length of rectangle”;
cin>>L;
cout<<”Enter breadth of rectangle”;
cin>>B;
}
int rectangle::area()
{
return L*B;
}
int main()
{
rectangle r;
r.get_data();
cout<<”Area of rectangle is:”<<r.area();
return 0;
}
```

```
#include<iostream>
using namespace std;

// Class definition
class Circle {
public:
    double radius;
    double  compute_area() {
        return 3.14*radius*radius;
    }

};

int main() {
    Circle obj;

    // Accessing public members outside class
    obj.radius = 5.5;
    cout << "Radius is: " << obj.radius << "\n";
    cout << "Area is: " << obj.compute_area();

    return 0;
}
```

Class Implementation

Example 1:

```
class MyClass {       // The class
  public:             // Access specifier
    int myNum;        // Attribute (int variable)
    string myString;  // Attribute (string variable)
};

int main() {
  MyClass myObj;  // Create an object of MyClass

  // Access attributes and set values
  myObj.myNum = 15;
  myObj.myString = "Some text";

  // Print attribute values
  cout << myObj.myNum << "\n";
  cout << myObj.myString;
  return 0;
}
```

Example 2:

```
// Create a Car class with some attributes
class Car {
  public:
    string brand;
    string model;
    int year;
};

int main() {
  // Create an object of Car
  Car carObj1;
  carObj1.brand = "BMW";
  carObj1.model = "X5";
  carObj1.year = 1999;

  // Create another object of Car
  Car carObj2;
  carObj2.brand = "Ford";
  carObj2.model = "Mustang";
  carObj2.year = 1969;

  // Print attribute values
  cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";
  return 0;
}
```

Example 3:

```
#include <iostream>
using namespace std;
class item
{
int number;    //private by default
float cost;      // private by default
public:
void getdata(int a, float b)
void putdata(void)
{
cout<<number<<endl;
cout<<cost<<endl;
}
};
void item :: getdata(int a, float b)
{
number = a;
cost = b;
}
int main()
{
item x;
cout<<“\n object x “<<endl;
x.getdata(200, 399.99);
x.putdata();

item y;
cout<<“\n object y”<<endl;
y.getdata(300,  400.00);
y.putdata();

}
```

Example 4:

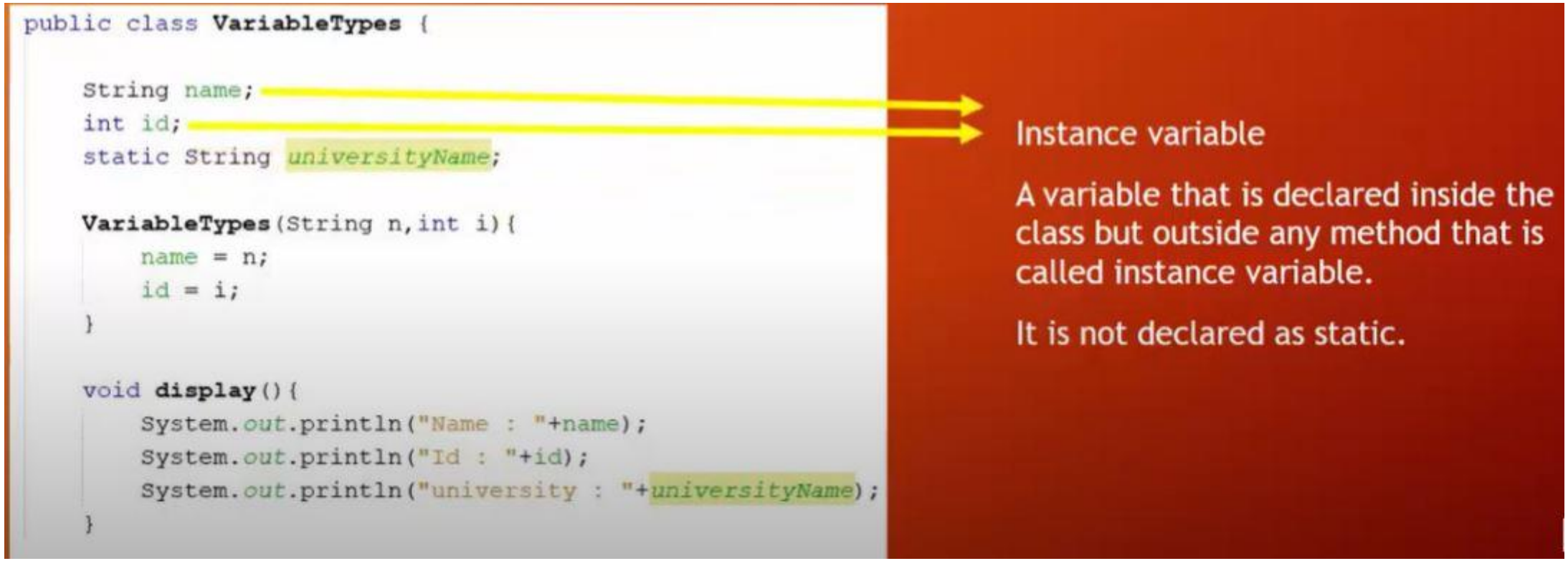
Write a program to read data of a student

```
#include<iostream>
using namespace std;
class student
{
private:
int roll;
char name[20];
public:
void getdata()
{cout<<"Enter Roll number:";
cin>>roll ;
cout<<"Enter Name:";
cin>>name;
}
void putdata()
{cout<<"Roll no:"<<roll<<endl;
cout<<Name:"<<name<<endl;
}
};
int main()
{
student s;
s.getdata();
s.putdata();
return 0;
}
```

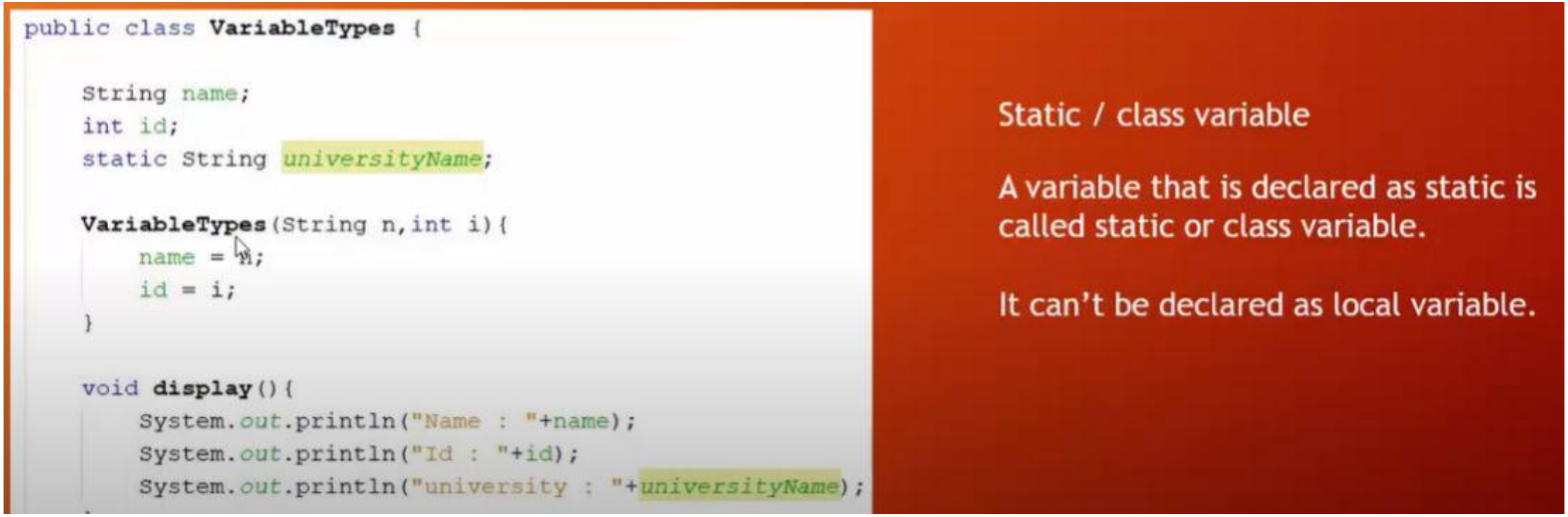
Local Variable



Instance Variable



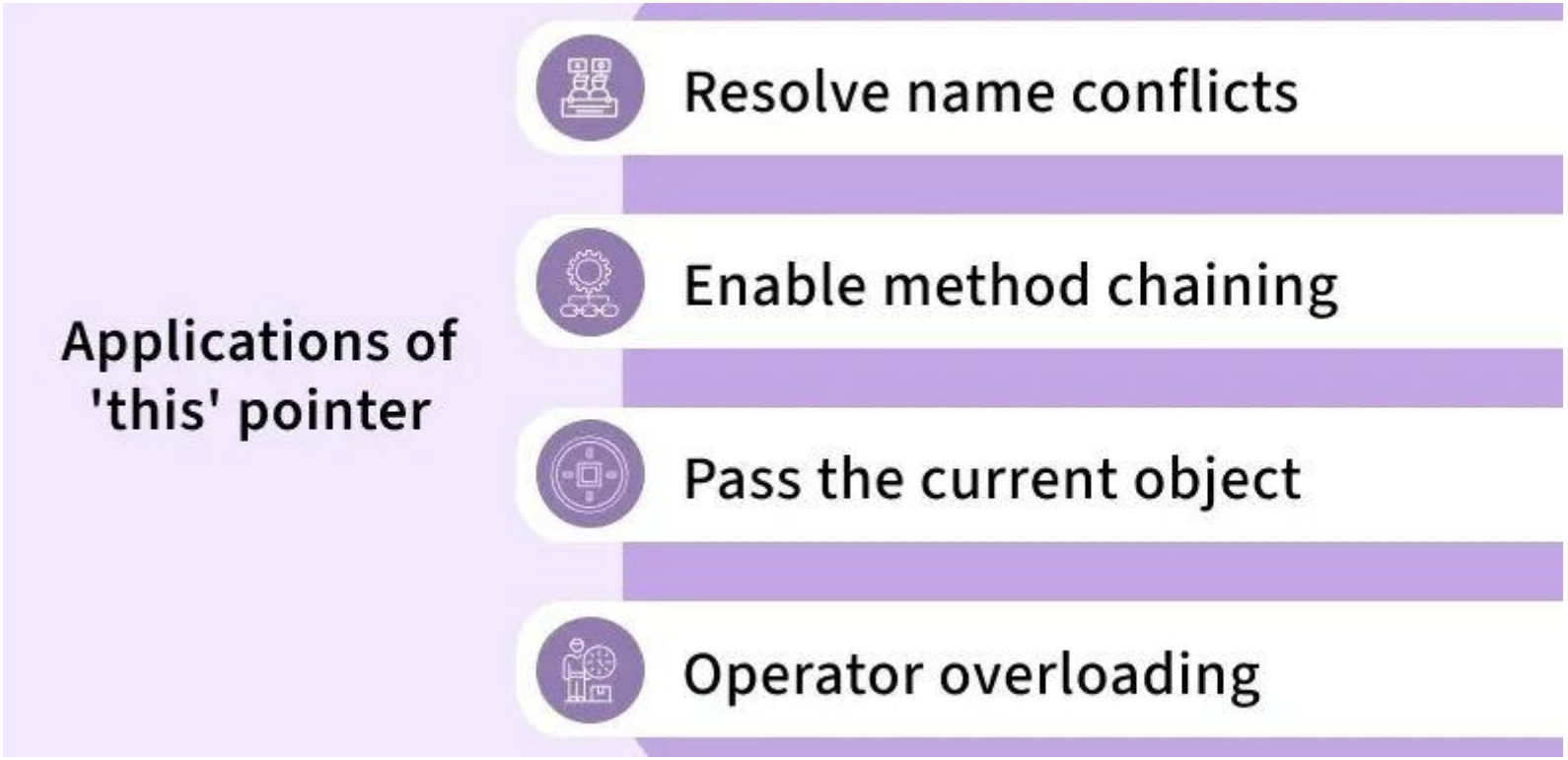
Class Variable



This Keyword

- **this** pointer helps in preventing self-assignment in assignment operators making sure that an object doesn’t assign itself to itself.
- **this** pointer makes method chaining possible by returning the current object (usually through *this), allowing you to call several methods on the same object in a single line of code.
- It allows for direct access to the object’s members within member functions.
- It is important in copy constructors and assignment operators, as it helps return the current object during assignments.
- it is also useful in Polymorphism, Inherited Classes, and Implement Fluent Interfaces, allowing smooth method chaining.

Application



C++ this Pointer

In C++ programming, **this** is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

- It can be used **to pass current object as a parameter to another method.**
- It can be used **to refer current class instance variable.**
- It can be used **to declare indexers.**

C++ this Pointer Example

```
#include <iostream>
using namespace std;
class Employee {
  public:
     int id; //data member (also instance variable)
     string name; //data member(also instance variable)
     float salary;
```

```
Employee(int id, string name, float salary)

  {
      this->id = id;
     this->name = name;
     this->salary = salary;
  }
```

This Keyword

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;

            void input(int id, double cgpa)
            {
                this->id = id;
                this->cgpa = cgpa;
            }

            void display()
            {
                cout << id << " " << cgpa << endl;
            }
    };
```

```cpp
int main()
{
    Student alif,esha;

    alif.input(101,3.80);
    alif.display();

    esha.input(102,3.90);
    esha.display();

    return 0;
}
```

```
void display()
        {
           cout<<id<<"  "<<name<<"  "<<salary<<endl;
        }
};
int main(void) {
    Employee e1 =Employee(101, “Karim", 890000); //creating an object of Employee
    Employee e2=Employee(102, “Addya", 59000); //creating an object of Employee
    e1.display();
    e2.display();
    return 0;
}
```

Output:

101 Karim 890000
102 Addya 59000